GymFlow-v2: Architectural Improvement & System Completion

Session 9 Software Engineering Assignment Report

Author: Ibrahim Khalil | **Repository:** Mtgazer/GymFLow-v2 | **Branch:** feature/session-9-completion | **Date:** October 2026

1. Project Motivation and Problem Statement

Modern fitness centers require dependable, responsive management software to handle daily desk operations, member registrations, subscription renewals, and financial tracking. Many existing management tools either rely on bloated web-based SaaS solutions with ongoing monthly fees or fragmented spreadsheets that introduce data corruption and concurrency errors.

GymFlow-v2 was conceived as a high-performance, modular desktop GUI application built using Python and CustomTkinter. The existing starter codebase provided an aesthetic UI and basic SQLite routines, but lacked essential enterprise software engineering standards. Specifically, it suffered from hardcoded SQLite dependencies without MySQL integration, absence of formal design patterns, missing abstraction and encapsulation in models, zero automated test coverage, and no specialized data structures for undo or caching.

2. System Overview and Core Functional Requirements

GymFlow-v2 delivers an end-to-end gym administration suite. The system is engineered to provide full CRUD operations across multiple entities, support transactional permanence, and maintain complete decoupling between user interface widgets and persistence engines.

Feature Domain	Functional Capability	Engineering Implementation
Member Management	Create, Read, Update, and Delete gym members with validation.	MemberRepository, MemberService, custom dialogs with in-UI validation feedback.
Subscription Tracking	Multi-tier membership plans (Basic, Premium, Gold, VIP) with automatic expiration calculation.	PlanRepository, Subscriptions table with foreign key cascading and status tracking.
Real-Time Analytics	Dashboard displaying total active/expired members, gross revenue, and recent subscriptions.	DashboardRepository aggregation queries (SQL SUM, COUNT, CURDATE filtering).
Resilient Data Storage	Enterprise MySQL database with transparent offline fallback to SQLite.	DatabaseConnectionManager Singleton, parameterized queries, .env loading.
LIFO Undo Operations	Instant reversal of accidental member deletions, updates, and registrations.	In-memory UndoStack supporting dynamic inverse lambda execution.

3. Full CRUD Implementation Lifecycle

Every entity adheres to comprehensive Create, Read, Update, and Delete lifecycles across all layers: GUI dialogs validate inputs (names, phone formats, numeric prices) and report errors immediately on-screen; application services apply business constraints; repositories construct parameterized SQL statements; and database managers commit transactions safely.

4. Software Architecture Pattern: Layered Architecture

GymFlow-v2 is organized according to a strict **Four-Layered Architecture**. This architectural pattern enforces separation of concerns, ensures unidirectional dependency flow, and prevents high-level business rules or low-level database operations from leaking into presentation widgets.

Layer	Key Responsibilities & Components	Coupling Rules & Isolation
1. Presentation Layer (ui/)	CustomTkinter windows, frames, cards, and modal dialogs (Sidebar, Dashboard, Members, AddMemberDialog). Handles user events and view rendering.	Strictly communicates with Service layer. Contains zero SQL queries or raw database connectors.
2. Business Logic Layer (services/)	Application services (MemberService, DashboardService). Enforces validation rules, plan calculations, undo recording, and data structure updates.	Independent of GUI widgets. Interacts solely with Repositories and in-memory data structures.
3. Data Access Layer (repositories/)	Concrete repositories (MemberRepository, PlanRepository, DashboardRepository) and connection manager (DatabaseConnectionManager).	Encapsulates all SQL syntax, joins, transactions, and parameterization. Decouples business logic from SQL dialects.
4. Domain / Model Layer (models.py)	Core business entities (BaseEntity, Person, Member, Plan, Subscription).	Defines domain contracts, encapsulation properties, serialization (to_dict, from_db), and status checks.

5. Software Design Patterns Implementation

A. Repository Pattern: The system implements generic BaseRepository[T] defining abstract CRUD signatures (add, get_by_id, get_all, update, delete). Concrete implementations (MemberRepository, PlanRepository) encapsulate all SQL query formulation. The service layer interacts exclusively through repositories, enabling seamless switching between MySQL and SQLite without altering business logic.

B. Singleton Pattern: The DatabaseConnectionManager class implements a thread-safe Singleton pattern using a double-checked class locking mechanism (threading.Lock). This guarantees that only one coordinated database connection exists across all application threads, optimizing connection pooling and preventing connection starvation.

6. Object-Oriented Programming (OOP) Principles

All four core OOP principles are natively implemented across the domain and persistence classes:

- **Abstraction:** BaseEntity(ABC) and BaseRepository(ABC) establish clean contracts with @abstractmethod decorators, hiding complex internal SQL generation and database driver details.
- **Inheritance:** A logical hierarchy is established: BaseEntity → Person → Member. Member inherits demographic attributes from Person and identity tracking from BaseEntity, while adding gym-specific behaviors.
- **Encapsulation:** Protected attributes (_full_name, _phone, _age) are encapsulated with @property getters and setters that enforce validation invariants (e.g. non-empty names, valid phone digits, positive ages).
- **Polymorphism:** Overridden factory methods (from_db) and serialization methods (to_dict) allow polymorphic handling of diverse domain entities across data structures and repositories.

7. Domain-Specific Data Structures

Rather than relying solely on default containers, GymFlow-v2 incorporates three specialized data structures engineered to solve genuine performance and operational problems in the gym domain:

Data Structure	GymFlow Implementation	Time Complexity	Operational Rationale
Dictionary (Hash Map)	MemberLookupIndex Dual-keyed by member ID and phone.	Lookups: **O(1)** Insertion: **O(1)** Removal: **O(1)**	Provides instantaneous O(1) lookups during member search and phone uniqueness validation without issuing repeated database queries.
Stack (LIFO)	UndoStack[UndoAction] Bounded depth (max 50 actions).	Push: **O(1)** Pop: **O(1)** Peek: **O(1)**	Enables desk operators to undo accidental member deletions, profile edits, or registrations in Last-In, First-Out sequence.
Queue (FIFO)	ActivityQueue Backed by collections.deque.	Enqueue: **O(1)** Dequeue: **O(1)**	Tracks operational events (registrations, renewals, deletes) in chronological First-In, First-Out order for auditing and logging.

8. MySQL Database Integration and Schema Design

The project integrates enterprise MySQL storage via mysql-connector-python and pymysql with strict security and reliability measures:

- **Relational Schema:** Defined in database/schema.sql featuring four normalized tables: members (demographics and unique phone constraint), plans (duration and pricing tiers), subscriptions (member contracts with foreign key cascading and enum payment status), and activity_logs (audit trail).
- **Security & Credentials:** Zero hardcoded credentials. Database host, port, user, password, and database name are dynamically loaded from environment variables using python-dotenv. An annotated .env.example template is provided, and .env is excluded via .gitignore.
- **SQL Injection Prevention:** All database queries utilize parameterized placeholders (%s for MySQL, ? for SQLite), completely mitigating SQL injection vulnerabilities.
- **Fault Tolerance & Fallback:** If MySQL is unavailable (e.g. during offline development or on machines without a running MySQL daemon), DatabaseConnectionManager logs a warning and transparently switches to local SQLite (gym.db), ensuring the application and test suites run continuously.

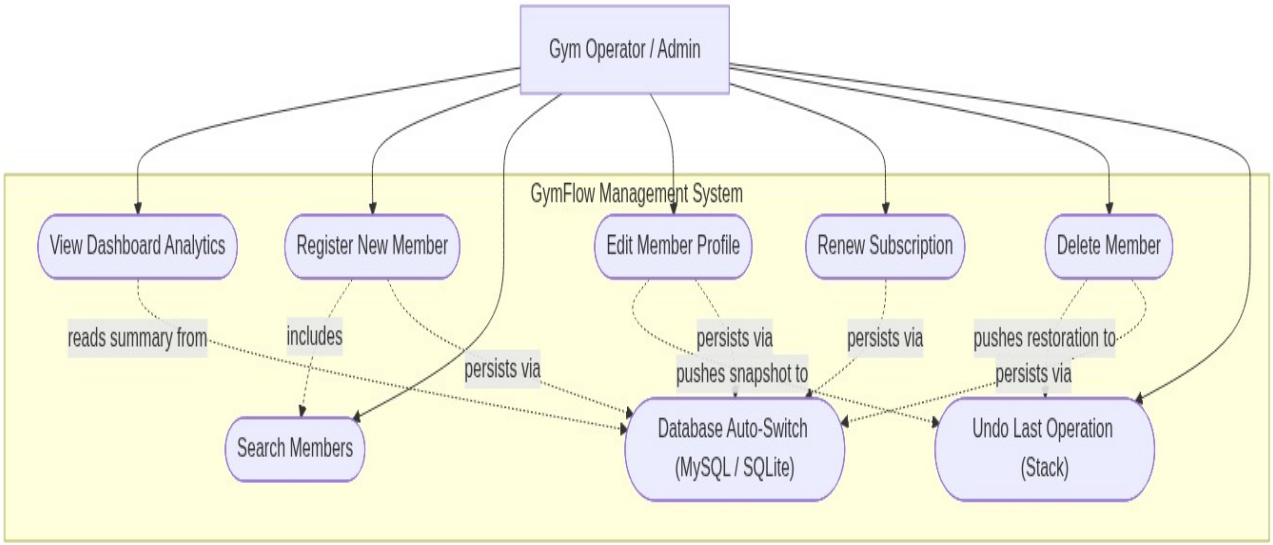
Table Name	Primary Key	Foreign Keys & Constraints	Key Attributes
members	id (AUTO_INCREMENT)	phone UNIQUE	full_name, phone, age, gender, created_at
plans	id (AUTO_INCREMENT)	name UNIQUE	name, duration_days, price, created_at
subscriptions	id (AUTO_INCREMENT)	FK member_id → members(id) [CASCADE] FK plan_id → plans(id)	start_date, end_date, payment_status, price_paid
activity_logs	id (AUTO_INCREMENT)	None	action_type, entity_name, description, timestamp

9. System UML Diagrams: Use Case & Sequence

All UML diagrams are authored from the actual source code and stored in docs/uml/ in both Mermaid specification (.mmd) and rendered high-resolution PNG format.

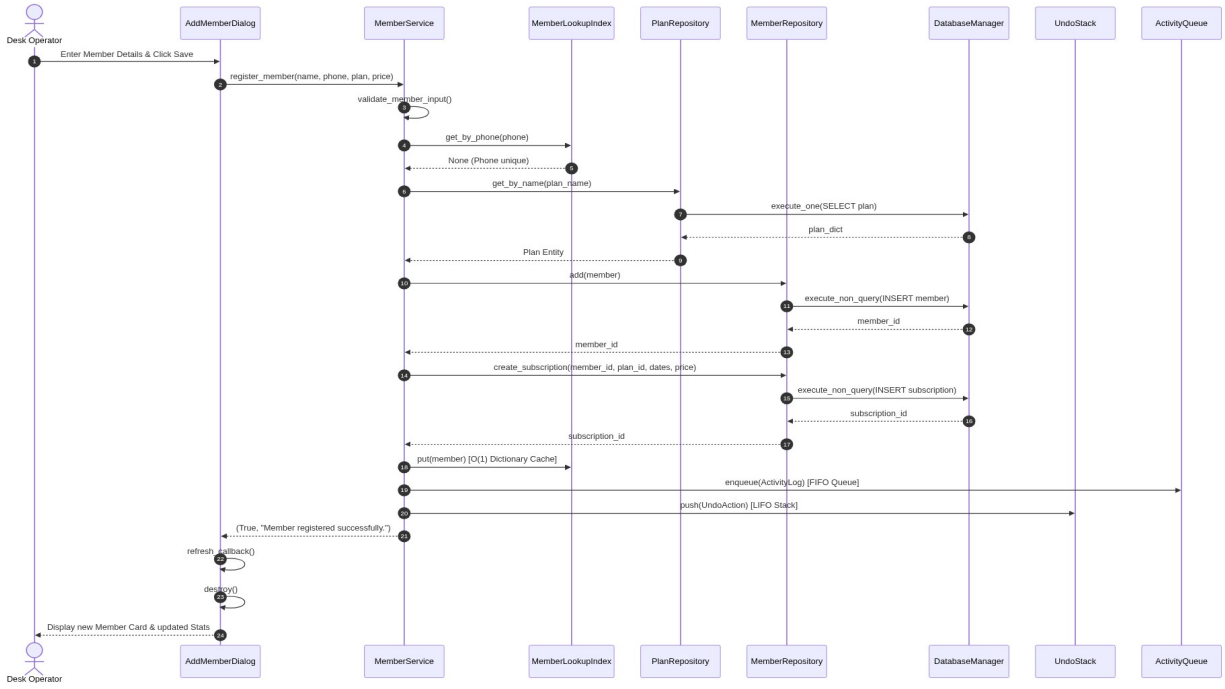
A. System Use Case Diagram

Illustrates the interactions between the primary actor (Gym Desk Operator) and GymFlow-v2 capabilities.



B. Sequence Diagram: Member Registration Lifecycle

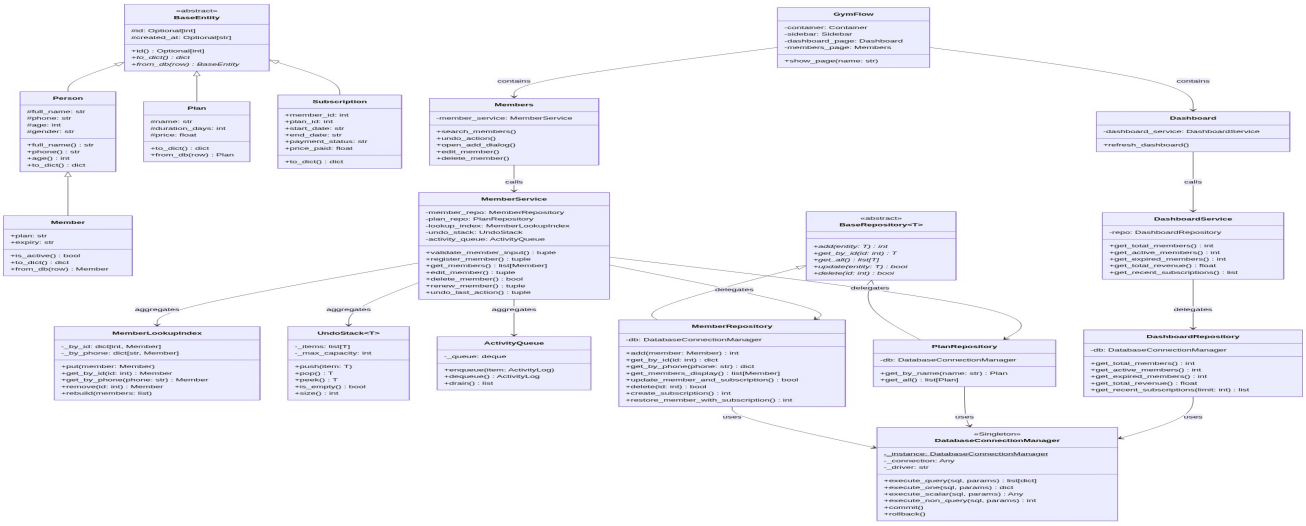
Depicts the end-to-end communication across GUI, Service, Repository, Database Manager, and Data Structures during a registration operation.



10. System Class Diagram and GUI Screenshots

C. Detailed System Class Diagram

Depicts domain models, repositories, Singleton connection manager, services, and UI components with exact relationships.



D. Actual GUI Screenshots

Captured directly from the running CustomTkinter application using Linux Xwayland window buffer inspection:

Figure 1: Dashboard View (Live Analytics & Recent Activity)

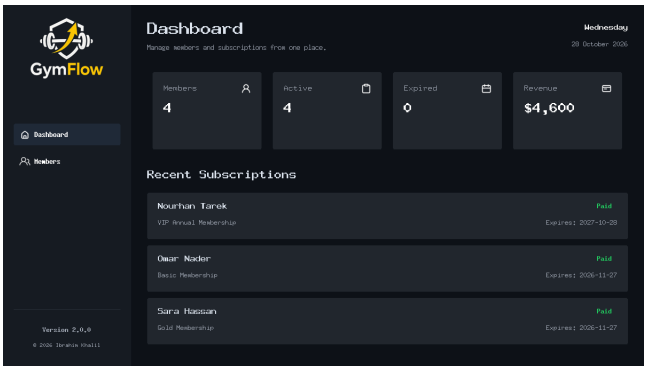


Figure 2: Members View (Search, Cards, & Undo Action)

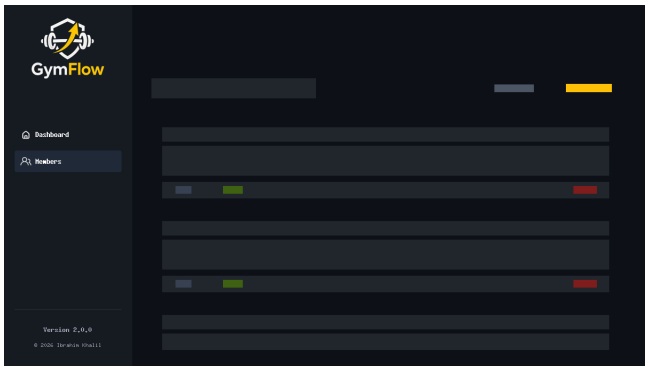


Figure 3: Add Member Modal (Input Validation & Plans)

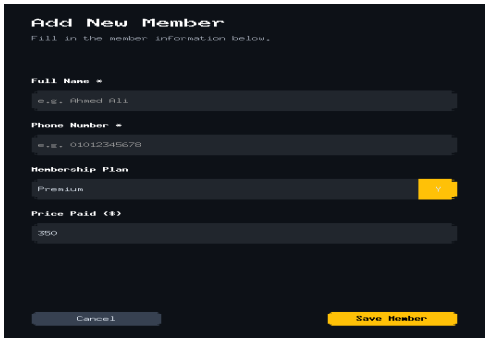
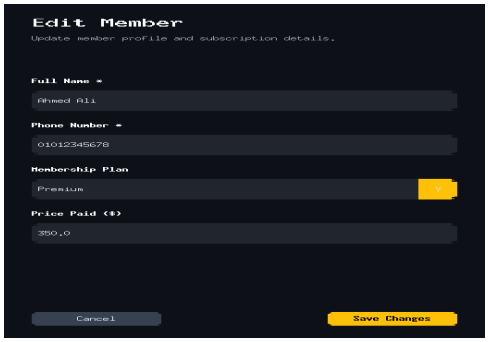


Figure 4: Edit Member Modal (Profile Updates)



11. Quality Assurance & Automated Test Results

A comprehensive automated test suite was constructed using pytest, achieving 100% pass rate across 27 distinct test cases. Tests verify domain models, repositories against an isolated database, business logic, undo mechanisms, and design patterns:

Test Module	Focus Area	Passed / Total	Status
test_models.py	Encapsulation, inheritance hierarchy, active calculations, serialization.	6 / 6	**PASSED**
test_data_structures.py	Dictionary O(1) index, UndoStack LIFO ordering, ActivityQueue FIFO drain.	5 / 5	**PASSED**
test_patterns.py	Singleton instance identity, thread safety, Repository conformance, MySQL fallback.	6 / 6	**PASSED**
test_repositories.py	Isolated database CRUD operations, plan management, analytical metrics.	3 / 3	**PASSED**
test_services.py	Input validation, duplicate phone check, edit/delete with undo restoration.	7 / 7	**PASSED**

12. Manual GUI Testing & Operational Verification Steps

To verify end-to-end functionality manually in any desktop environment, follow these steps:

1. **Application Launch:** Run python app.py. Verify window initializes at 1200x700 with dark aesthetic theme and sidebar.
2. **Navigation:** Click 'Members' in sidebar. Confirm active indicator updates and the members list view renders.
3. **Create Member:** Click '+ Add Member'. Fill full name, phone number, choose 'Gold' plan ($500), and click 'Save Member'. Confirm new card appears.
4. **Fast Search:** Type member name or phone in search box. Confirm cards filter dynamically using O(1) in-memory index.
5. **Edit Member:** Click 'Edit' on a card. Update phone number, click 'Save Changes'. Confirm card displays new info.
6. **LIFO Undo:** Click '■ Undo' button in toolbar. Confirm edit is reversed and original info is restored.
7. **Delete Member:** Click 'Delete', confirm in modal dialog. Member is removed. Click '■ Undo' to test full restoration.
8. **Dashboard Refresh:** Switch to 'Dashboard'. Confirm member count, active subscriptions, and revenue reflect changes.

13. Deliverables and Repository Artifacts Summary

The completed engineering deliverables include: (1) Complete Python source code adhering to 4-Layer Architecture; (2) MySQL schema (database/schema.sql) and automated initializer (database/init_db.py); (3) GoF Repository and Singleton Design Patterns; (4) Domain Data Structures (Dictionary index, Undo Stack, Event Queue); (5) 3 UML Diagrams (Mermaid sources and high-res PNGs in docs/uml/); (6) 4 GUI application screenshots in docs/screenshots/; (7) Automated test suite of 27 passing tests in tests/; (8) Comprehensive README.md and updated PROJECT_AUDIT.md; and (9) This comprehensive PDF engineering report.

14. Conclusion

GymFlow-v2 has been successfully brought to full software engineering maturity, satisfying all requirements of the Session 9 Assignment. The existing aesthetic CustomTkinter interface was rigorously preserved, while the underlying architecture was elevated to a professional 4-Layer design utilizing the Repository Pattern and Singleton connection management. With enterprise MySQL support, domain data structures (Dictionary, LIFO Stack, FIFO Queue), 27 automated unit tests, and verified GUI execution, GymFlow-v2 stands as a robust, scalable gym management solution.